

Usando Bison

O capítulo anterior concentrou-se apenas no Flex. Neste capítulo, voltamos nossa atenção para o Bison, embora usemos o Flex para gerar nossos analisadores léxicos. Enquanto o Flex reconhece expressões regulares, o Bison reconhece gramáticas inteiras. O Flex divide o fluxo de entrada em partes (tokens), e então o Bison pega essas partes e as agrupa logicamente. Neste capítulo, terminaremos a calculadora de mesa que começamos no [Capítulo 1](#), começando com aritmética simples e adicionando funções integradas, variáveis de usuário e, finalmente, funções definidas pelo usuário.

Como um analisador sintático Bison compara sua entrada

O Bison recebe uma gramática que você especifica e escreve um analisador sintático que reconhece "sentenças" válidas nessa gramática. Usamos o termo "sentença" aqui de uma forma bastante geral — para uma gramática da linguagem C, as sentenças são programas em C sintaticamente válidos. Os programas podem ser sintaticamente válidos, mas semanticamente inválidos; por exemplo, um programa em C que atribui uma string a uma variável inteira. O Bison lida apenas com a sintaxe; outras validações ficam a seu critério. Como vimos no [Capítulo 1](#), uma gramática é uma série de regras que o analisador sintático usa para reconhecer entradas sintaticamente válidas. Por exemplo, aqui está uma versão da gramática que usaremos mais adiante neste capítulo em uma calculadora:

declaração: NOME '=' expressão

expressão: NÚMERO '+' NÚMERO
 | NÚMERO '-' NÚMERO

A barra vertical, |, significa que existem duas possibilidades para o mesmo símbolo; ou seja, uma expressão pode ser uma adição ou uma subtração. O símbolo à esquerda dos dois pontos (:) é conhecido como o lado esquerdo da regra, frequentemente abreviado como LHS, e os símbolos à direita são o lado direito, geralmente abreviado como RHS. Várias regras podem ter o mesmo lado esquerdo; a barra vertical é apenas uma abreviação para isso. Os símbolos que aparecem na entrada e são retornados pelo analisador léxico são símbolos terminais ou tokens, enquanto aqueles que aparecem no lado esquerdo de cada regra são símbolos não terminais ou não terminais.

Os símbolos terminais e não terminais devem ser diferentes; é um erro escrever uma regra com um token no lado esquerdo.

A maneira usual de representar uma frase analisada é como uma árvore. Por exemplo, se analisássemos a entrada `fred = 12 + 13` com esta gramática, a árvore teria a aparência da [Figura 3-1](#).

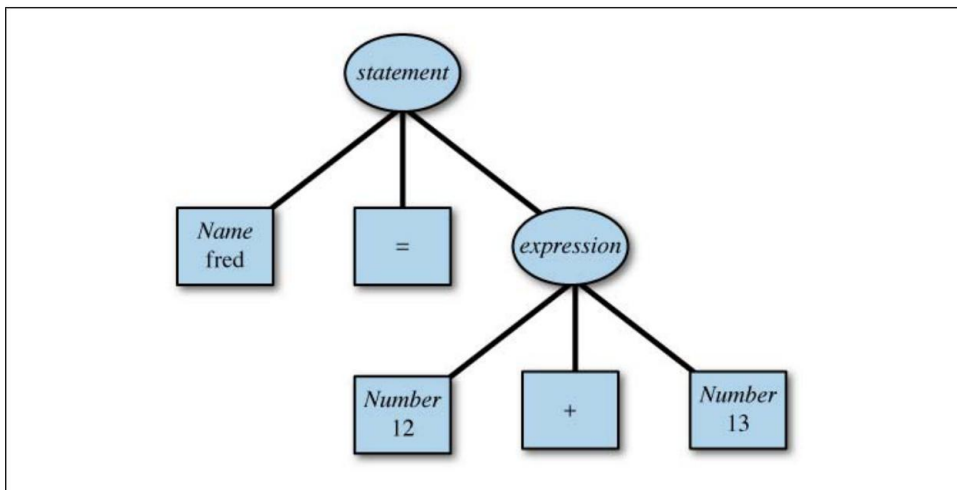


Figura 3-1. Árvore de análise sintática da expressão

Neste exemplo, `12 + 13` é uma expressão e `fred = expressão` é uma declaração. Um analisador sintático Bison não cria automaticamente essa árvore como uma estrutura de dados, embora, como veremos, não seja difícil fazê-lo manualmente. Toda gramática inclui um símbolo inicial, aquele que deve estar na raiz da árvore de análise sintática. Nesta gramática, declaração é o símbolo inicial.

As regras podem se referir a si mesmas direta ou indiretamente; essa importante capacidade possibilita a análise de sequências de entrada arbitrariamente longas. Vamos estender nossa gramática para lidar com expressões aritméticas mais longas:

```

expressão: NÚMERO
          | expressão + NÚMERO |
          expressão ÷ NÚMERO
  
```

Agora podemos analisar uma sequência como `fred = 14 + 23 ÷ 11 + 7` aplicando as regras de expressão repetidamente, como na [Figura 3-2](#). O Bison consegue analisar regras recursivas com muita eficiência, então veremos regras recursivas em quase todas as gramáticas que usarmos.

Análise sintática Shift/Reduce

Um analisador sintático Bison funciona procurando regras que possam corresponder aos tokens vistos até o momento. Quando o Bison processa um analisador sintático, ele cria um conjunto de estados, cada um refletindo uma possível posição em uma ou mais regras parcialmente analisadas. À medida que o analisador lê os tokens, cada vez que lê um token que não completa uma regra, ele o empilha em uma pilha interna.

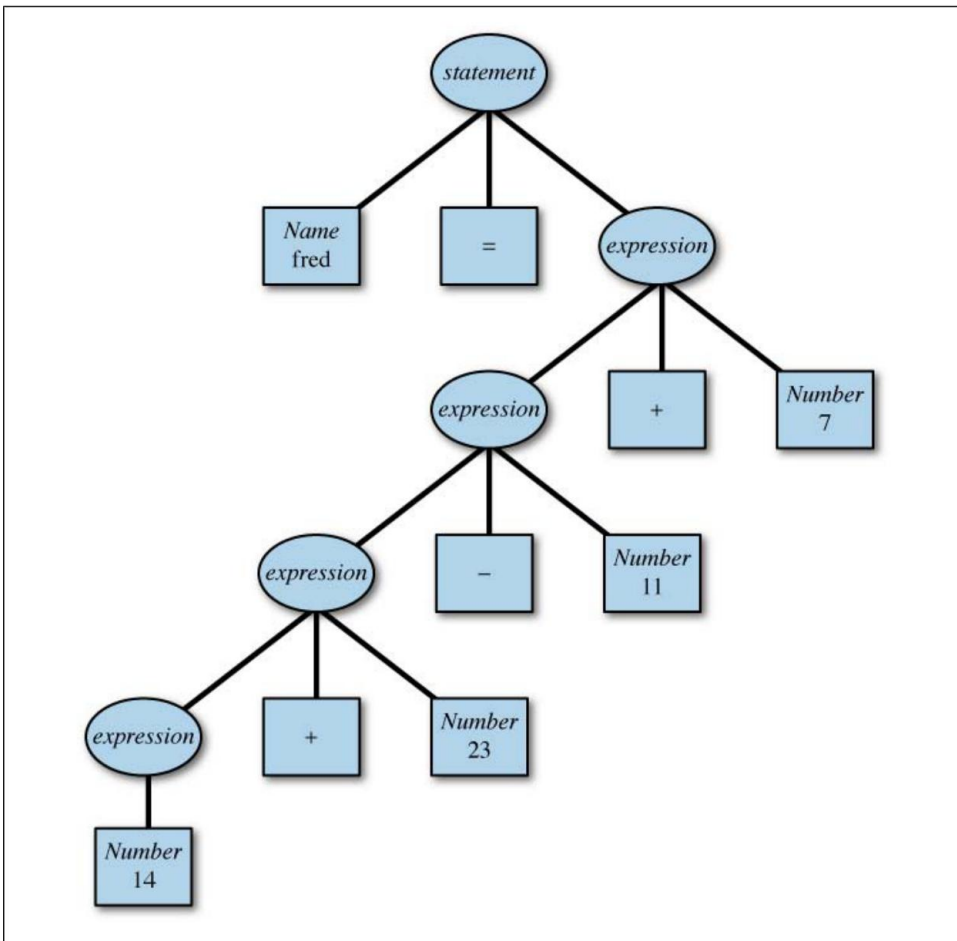


Figura 3-2. Árvore de análise sintática com regras recursivas

O sistema muda para um novo estado que reflete o token que acabou de ler. Essa ação é chamada de mudança de estado. Quando encontra todos os símbolos que constituem o lado direito de uma regra, ele remove os símbolos do lado direito da pilha, coloca o símbolo do lado esquerdo na pilha e muda para um novo estado que reflete o novo símbolo na pilha. Essa ação é chamada de redução, pois geralmente reduz o número de itens na pilha.* Sempre que o Bison reduz uma regra, ele executa o código do usuário associado à regra. É assim que você realmente faz algo com o material que o analisador sintático analisa. Vejamos como ele analisa a entrada `fred = 12 + 13` usando as regras simples na [Figura 3-1](#). O analisador sintático começa movendo os tokens para a pilha interna um de cada vez:

* É possível ter regras com lados direitos vazios, caso em que uma redução termina com um item a mais do que tinha inicialmente, mas mesmo assim as chamamos de reduções.

```
fred
fred =
fred = 12 fred
= 12 + fred = 12
+ 13
```

Neste ponto, ele pode reduzir a expressão da regra: NÚMERO + NÚMERO, então ele remove o 12, o sinal de mais e o 13 da pilha e os substitui pela *expressão*:

```
fred = expressão
```

Agora, a regra é reduzida : NOME = expressão, então ela remove fred, = e expressão e os substitui por *declaração*. Chegamos ao final da entrada e a pilha foi reduzida ao símbolo inicial, então a entrada era válida de acordo com a gramática.

O que o analisador LALR(1) do Bison não consegue

analisar Os analisadores do Bison podem usar um dos dois métodos de análise sintática, conhecidos como LALR(1) (Look Ahead Left to Right com um token de antecipação) e GLR (Generalized Left to Right). A maioria dos analisadores usa LALR(1), que é menos poderoso, mas consideravelmente mais rápido e fácil de usar do que o GLR. Neste capítulo, descreveremos a análise sintática LALR e deixaremos o GLR para o [Capítulo 9](#).

Embora a análise sintática LALR seja bastante poderosa, você pode escrever gramáticas que ela não consegue processar. Ela não consegue lidar com gramáticas ambíguas, aquelas em que a mesma entrada pode corresponder a mais de uma árvore de análise sintática. Ela também não consegue lidar com gramáticas que precisam de mais de um token de lookahead para determinar se correspondeu a uma regra. (Mas o Bison tem uma solução excelente para lidar com o tipo mais comum de gramáticas ambíguas, que abordaremos em algumas páginas.) Considere este exemplo extremamente artificial: frase: cart_animal AND CART | work_animal AND PLOW

```
animal_carteira: CAVALO | CABRA
```

```
animal_de_trabalho: CAVALO | BOI
```

Essa gramática não é ambígua, já que existe apenas uma árvore de análise sintática possível para qualquer entrada válida, mas o Bison não consegue lidar com ela porque requer dois símbolos de lookahead. Em particular, na entrada HORSE AND CART, ele não consegue determinar se HORSE é um animal de carroça ou um animal de trabalho até encontrar CART, e o Bison não consegue olhar tão à frente. Se alterássemos a primeira regra para:

```
frase: animal_de_carga CARRINHO |
      animal_de_trabalho ARADO
```

O Bison não teria problemas, já que consegue prever um token para verificar se a entrada "HORSE" é seguida por "CART", caso em que o cavalo é um animal de carroça, ou por "PLOW", caso em que é um animal de trabalho. Na prática, as regras sobre o que o Bison consegue processar não são tão complexas e confusas quanto podem parecer aqui. Uma das razões é que o Bison sabe exatamente quais gramáticas consegue analisar e quais não consegue. Se você fornecer uma gramática que ele não consegue analisar, ele não conseguirá.

O sistema irá te avisar, então não há problema de analisadores sintáticos excessivamente complexos falharem silenciosamente. Outro motivo é que as gramáticas que o Bison consegue processar correspondem bastante bem às gramáticas que as pessoas realmente escrevem. Muitas vezes, uma construção gramatical que confunde o Bison também confundirá as pessoas; portanto, se você tiver alguma liberdade no design da sua linguagem, considere alterá-la para torná-la mais compreensível tanto para o Bison quanto para seus usuários. Para mais informações sobre análise sintática shift/reduce, consulte o [Capítulo 7](#).

Para uma discussão sobre o que o Bison precisa fazer para transformar sua especificação em um programa C funcional, uma boa referência é o livro "Parsing Techniques: A Practical Guide" de Dick Grune. Ele oferece o download de uma edição antiga, mas bastante adequada, em <http://www.cs.vu.nl/~dick/PTAPG.html>.

Um analisador de bisão

Uma especificação Bison tem a mesma estrutura de três partes que uma especificação Flex. (O Flex copiou sua estrutura do Lex, que por sua vez copiou a estrutura do Yacc, o predecessor do Bison.) A primeira seção, a seção de definição, lida com informações de controle para o analisador sintático e geralmente configura o ambiente de execução no qual o analisador irá operar. A segunda seção contém as regras para o analisador sintático, e a terceira seção é o código C copiado literalmente para o programa C gerado.

O Bison cria o programa em C inserindo partes em um arquivo esqueleto padrão. As regras são compiladas em matrizes que representam a máquina de estados correspondente aos tokens de entrada.

As ações têm os valores \$N e @N traduzidos para C e, em seguida, são colocados em uma instrução switch dentro de `yyparse()`, que executa a ação apropriada sempre que ocorre uma redução. Algumas partes do esqueleto têm múltiplas versões, das quais o bison escolhe dependendo das opções em uso; por exemplo, se o analisador sintático usa o recurso de localizações, ele inclui código para lidar com dados de localização.

Neste capítulo, pegamos o exemplo simples da calculadora do [Capítulo 1](#) e o expandimos significativamente. Primeiro, reescrevemos o exemplo para aproveitar alguns atalhos úteis do Bison e o modificamos para produzir uma estrutura de dados reutilizável, em vez de calcular os valores dinamicamente. Mais tarde, adicionaremos uma sintaxe mais complexa para loops e funções e mostraremos como implementá-los em um interpretador simples.†

Árvores de Sintaxe Abstrata

Uma das estruturas de dados mais poderosas usadas em compiladores é a árvore sintática abstrata (AST). No [Capítulo 1](#), vimos uma árvore de análise sintática, uma árvore que possui um nó para cada regra usada para analisar a string de entrada. Na maioria das gramáticas reais, existem regras que servem para gerenciar o agrupamento, mas que não adicionam significado ao programa. No exemplo da calculadora, as regras

† Então vamos parar por aqui; o mundo já tem muitos programas interpretadores de scripts. Para fins práticos, você estará muito melhor adicionando algumas extensões a uma linguagem de script já existente e bem depurada, como Python, Perl ou Lua, em vez de escrever mais uma.

exp: term e term: factor existem apenas para informar ao analisador sintático a precedência relativa dos operadores. Uma AST é basicamente uma árvore de análise sintática que omite os nós das regras irrelevantes.

Uma vez que um analisador sintático cria uma AST (Árvore Sintática Abstrata), é simples escrever rotinas recursivas que "percorrem" a árvore. Veremos vários desses "percorredores de árvore" neste exemplo.

Uma calculadora aprimorada que cria ASTs. Este exemplo é grande o suficiente

para valer a pena dividi-lo em vários arquivos de origem, então colocaremos a maior parte do código C em um arquivo separado, o que significa que também precisamos de um arquivo de cabeçalho C para declarar as rotinas e estruturas de dados usadas nos vários arquivos ([Exemplo 3-1](#)).

Exemplo 3-1. Calculadora que constrói uma AST: cabeçalho fb3-1.h

```
/*
 * Declarações para uma calculadora fb3-1 */

/* interface para o analisador léxico
 */ extern int yylineno; /* do analisador léxico */
void yyerror(char *s, ...);

/* nós na árvore de sintaxe abstrata */ struct ast
{ int nodetype;
  struct ast *l;
  struct ast *r; };

struct numval { int
  nodetype; /* tipo K para constante */ double number; };

/* construir uma AST */
struct ast *newast(int nodetype, struct ast *l, struct ast *r); struct ast *newnum(double
d);

/* avaliar uma AST */
double eval(struct ast *);

/* exclui e libera uma AST */ void
treefree(struct ast *);
```

A variável yylineno e a rotina yyerror são familiares do exemplo flex. Nossa função yyerror foi ligeiramente aprimorada para aceitar múltiplos argumentos no estilo de printf.

A AST consiste em nós, cada um com um tipo de nó. Nós diferentes possuem campos diferentes, mas por enquanto temos apenas dois tipos: um que contém ponteiros para até dois subnós e outro que contém um número. Duas rotinas, `newast` e `newnum`, criam nós da AST;

A função `eval` percorre uma AST e retorna o valor da expressão que ela representa; e a função `treefree` percorre uma AST e exclui todos os seus nós.

Exemplo 3-2. Analisador sintático Bison para calculadora AST

```
/* calculadora com AST */

%
{ #include <stdio.h> #include
<stdlib.h> #include "fb3-1.h"
%}

%união
{ struct ast *a;
double d; }

/* declarar tokens */ %token
<d> NÚMERO %token
FIM DA LINHA

%tipo <a> termo fator exponencial
```

O Exemplo 3-2 mostra o analisador sintático bison para a calculadora AST. A primeira seção do analisador usa a construção %union para declarar os tipos a serem usados nos valores dos símbolos no analisador. Em um analisador bison, cada símbolo, tanto tokens quanto não terminais, pode ter um valor associado a ele. Por padrão, os valores são todos inteiros, mas programas úteis geralmente precisam de valores mais sofisticados. A construção %union, como o nome sugere, é usada para criar uma declaração de união em linguagem C para valores de símbolos. Neste caso, a união tem dois membros: a, que é um ponteiro para uma AST, e d, que é um número de dupla precisão.

Uma vez definida a união, precisamos informar ao Bison quais símbolos possuem quais tipos de valores, colocando o nome apropriado da união entre colchetes angulares (< >). O token NUMBER, que representa números na entrada, tem o valor <d> para armazenar o valor do número. A nova declaração %type atribui o valor <a> a exp, factor e term, que usaremos à medida que construirmos nossa AST.

Você não precisa declarar um tipo para um token nem declarar um não-terminal se não usar o valor do símbolo. Se houver uma `%union` nas declarações, o Bison retornará um erro se você tentar usar o valor de um símbolo que não possui um tipo atribuído. Lembre-se de que qualquer regra sem código de ação explícito recebe a ação padrão $\$\$ = \1 ;, e o Bison reclamará se o símbolo do lado esquerdo tiver um tipo e o símbolo do lado direito não tiver o mesmo tipo.

```
%%
calclist: /* nada */ | calclist exp
EOL { printf("= %4.4g\n",
eval($2)); avalia e imprime a AST treefree($2); libera a AST
```

```

        printf("> ");
    }

    | calclist EOL { printf("> "); } /* linha em branco ou um comentário */ ;

exp: fator | exp
    '+' fator { $$ = newast('+', $1,$3); } | exp '-' fator { $$ = newast('-',
    $1,$3); } ;

fator: termo
    | factor '*' term { $$ = newast('*', $1,$3); } | factor '/' term { $$ = newast('/',
    $1,$3); } ;

termo: NÚMERO { $$ = novonum($1); } | ']' termo { $
    $ = novoast('[', $2, NULL); } | '(' exp ')' { $$ = $2; } | '-' termo { $$ =
    novoast('M', $2, NULL); }

;
%%

```

Fichas de Caracteres Literais

A seção de regras apresenta duas mudanças significativas em relação à versão do [Capítulo 1](#). Uma delas é que as regras agora utilizam tokens literais para os operadores. Em vez de atribuir um nome a cada token, também é possível usar um caractere entre aspas simples como token, sendo o valor ASCII do token o seu número. (O Bison inicia a numeração dos tokens nomeados em 258, portanto não há problemas de colisão.) Por convenção, tokens de caracteres literais são usados para representar tokens de entrada compostos pelo mesmo caractere; por exemplo, o token '+' representa o token de entrada +, portanto, na prática, eles são usados apenas para pontuação e operadores. Não é necessário declarar tokens de caracteres literais, a menos que seja preciso declarar o tipo de seus valores.

A outra mudança no analisador sintático é que ele cria uma AST (Árvore Sintática Abstrata) para cada expressão, em vez de avaliá-la dinamicamente. Nesta versão da calculadora, criamos uma AST para cada expressão e, em seguida, avaliamos a AST, imprimimos o resultado e liberamos a AST. Chamamos `newast()` para criar cada nó na AST. Cada nó tem um tipo de operador, que geralmente é o mesmo que o nome do token. Observe que a subtração unária cria um nó do tipo M para distingui-la da subtração binária.

Exemplo 3-3. Analisador léxico para calculadora AST

```

/* reconhecer tokens para a calculadora */ %option noyywrap
nodefult yylineno %c #include "fb3-1.h" #include

"fb3-1.tab.h" %c

```



```

/* expoente de ponto
EXP      flutuante */ ([Ee][-+]?[0-9]+)

%
% "+"
| "-" |
| "." |
| "e" | "E" |
| "+" | "-" |
| "(" |
| ")"
{ return yytext[0]; }
[0-9]+ "." [0-9]* {EXP}? | "."? [0-9]+
{EXP}? { yylval.d = atof(yytext); return NUMBER; }

\n      { return EOL; }

"/*.* [\t]
      { /* ignorar espaços em branco */ }

.      { yyerror("Caractere misterioso %c\n", *yytext); }

%%

```

O analisador léxico, mostrado no [Exemplo 3-3](#), é um pouco mais simples do que a versão do [Capítulo 1](#). Usamos um padrão comum para os operadores de caractere único, tratando todos eles com a mesma regra que retorna yytext[0], o próprio caractere, como token. Ainda temos nomes para NUMBER e EOL. Também lidamos com números de ponto flutuante, usando uma versão do padrão do [Capítulo 2](#), e alteramos a representação interna dos números para double. Como yylval agora é uma união, o valor double precisa ser atribuído a yylval.d. (O Flex não automatiza o gerenciamento de valores de token como o Bison faz.)

Exemplo 3-4. Rotinas em C para calculadora AST

```

#include <stdio.h> #include
<stdlib.h> #include <stdarg.h>
#include "fb3-1.h"

struct ast *
newast(int nodetype, struct ast *l, struct ast *r) { struct ast *a =
malloc(sizeof(struct ast));

    if(la)
        { yyerror("sem espaço"); exit(0); }
    a->nodetype

    = nodetype; a->l = l; a->r = r;
    return a; }

struct ast *
newnum(double d)

{ struct numval *a = malloc(sizeof(struct numval));

```

```

if(!a)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= 'K'; a->number = d;
return (struct ast
*)a; }

```

Finalmente, há um arquivo de rotinas chamadas pelo analisador sintático, mostrado no [Exemplo 3-4.‡](#) Primeiro vêm as duas rotinas para criar nós da AST, alocando um nó com malloc e preenchendo-o. Todos os nós da AST têm um tipo de nó como primeiro campo, para que uma caminhada na árvore possa saber que tipo de nós está percorrendo.

```

double
eval(struct ast *a) { double

v; valor calculado desta subárvore

switch(a->nodetype) { case
'K': v = ((struct numval *)a)->number; break;

case '+': v = eval(a->l) + eval(a->r); break; case '-': v = eval(a->l)
- eval(a->r); break; case '*': v = eval(a->l) * eval(a->r); break;
case '/': v = eval(a->l) / eval(a->r); break; case '!': v = eval(a->l);
if(v < 0) v = -v; break; case 'M': v = -eval(a->l); break; default:
printf("erro interno: nó inválido %c\n", a->nodetype); } return v; }

```

```

vazio

treefree(struct ast *a) { switch(a-

>nodetype) {

/* duas subárvores */
caso '+':
caso '-':
caso '*':
caso '/':
treefree(a->r);

/* uma subárvore */
caso '!':
caso 'M':
treefree(a->l);

```

‡ Estes trechos poderiam ter sido incluídos na terceira seção do analisador sintático, mas é mais fácil depurar seu programa se você não adicionar muito código extra ao arquivo do analisador.

```

        /* nenhuma subárvore
*/ caso 'K':
    free(a);
    break;

padrão: printf("erro interno: liberar nó inválido %c\n", a->nodetype); } }

```

Em seguida, temos as duas rotinas de percurso na árvore. Cada uma delas realiza o que é conhecido como percurso em profundidade na árvore, visitando recursivamente as subárvores de cada nó e, em seguida, o próprio nó. A rotina `eval` retorna o valor da árvore ou subárvore de cada chamada, e a rotina `treefree` não precisa retornar nada.

```

void
yyerror(char *s, ...) { va_list
    ap; va_start(ap, s);

    fprintf(stderr, "%d: erro: ", yylineno); vfprintf(stderr, s, ap);
    fprintf(stderr, "\n"); }

int
main()

{ printf("> "); return
    yyparse(); }

```

Finalmente, temos yyerror e main. Esta versão de yyerror usa varargs para aceitar uma lista de argumentos no estilo printf, o que se mostra conveniente ao gerar mensagens de erro.

Construindo a Calculadora AST: Este

programa agora possui três arquivos de código-fonte e um arquivo de cabeçalho, então, é claro, usamos o comando make para compilá-lo.

```

fb3-1: fb3-1.l fb3-1.y fb3-1.h bison -d fb3-1.y
    flex -o fb3-1.lex.c fb3-1.l
    cc -o $@ fb3-1.tab.c fb3-1.lex.c
    fb3-1funcs.c

```

Observe a opção -o do flex. O Bison nomeia automaticamente o arquivo C gerado para corresponder ao arquivo .y, mas o flex sempre nomeia seu arquivo C como lex.yy.c, a menos que você especifique o contrário.

Conflitos de turno/redução e precedência do operador

O analisador de expressões usa três símbolos diferentes, `exp`, `factor` e `term`, para definir o precedência e associatividade dos operadores. Embora este analisador ainda seja razoavelmente legível, à medida que as gramáticas adicionam mais operadores com mais níveis de precedência, elas se tornam cada vez mais difíceis de ler e manter. Bison oferece uma maneira inteligente de descrever o precedência separadamente das regras da gramática, o que torna a gramática e

O analisador sintático ficará menor e mais fácil de manter. Primeiro, vamos fazer com que todas as expressões usem `exp` símbolos:

```
%type <a> exp

%%
...
exp: exp '+' exp { $$ = newast('+', $1,$3); }
    | exp '-' exp { $$ = newast('-', $1,$3); }
    | exp '*' exp { $$ = newast('*', $1,$3); }
    | exp '/' exp { $$ = newast('/', $1,$3); }
    | '(' exp ')' { $$ = newast('(', $2, NULL); }
    | '-' { $$ = newast('M', $2, NULL); }
    | NÚMERO { $$ = novonum($1); }
;
%%
```

Mas essa gramática tem um problema: ela é extremamente ambígua. Por exemplo, a entrada `2+3*4` pode significar $(2+3)*4$ ou $2+(3*4)$, e a entrada `3-4-5-6` pode significar $3-(4-(5-6))$ ou $(3-4)-(5-6)$ ou qualquer uma de muitas outras possibilidades. A Figura 3-3 mostra as duas possibilidades. analisa para `2+3*4`.

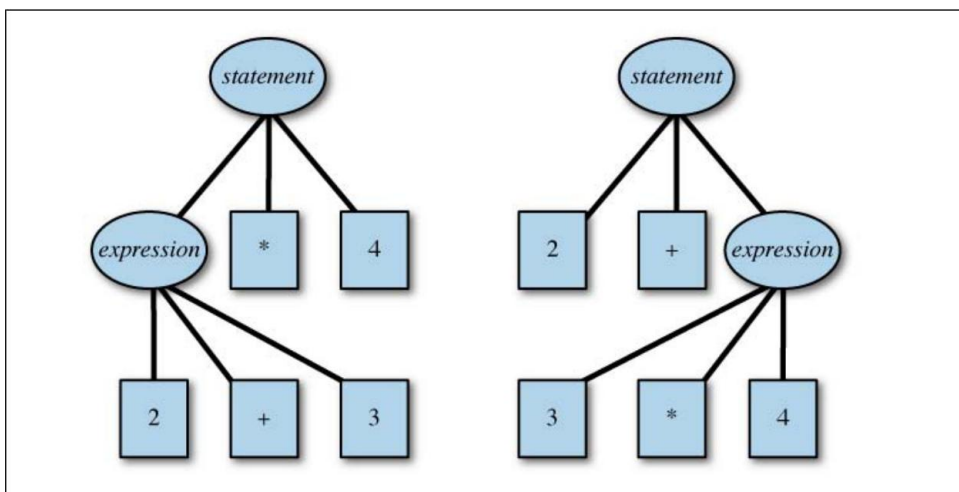


Figura 3-3. Duas árvores de análise sintática de expressões

Se você compilar essa gramática como está, o Bison informará que existem 24 operações de shift/reduce. conflitos, que são estados em que não é possível determinar se o token deve ser movido para o Empilhe ou reduza uma regra primeiro. Por exemplo, ao analisar $2+3*4$, o analisador percorre o seguinte caminho: estes passos (aqui abreviamos exp como E):

```

2          turno NÚMERO
E          reduzir a  NÚMERO
E +        mudança E +
E + 3      turno NÚMERO
E + E      reduzir E  NÚMERO

```

Neste ponto, o analisador examina o * e pode simplificar $2+3$ usando:

```
exp:      exp '+' exp
```

para uma expressão ou deslocamento do *, esperando poder reduzir:

```
exp:      exp '*' exp
```

mais tarde.

O problema é que não contamos aos bisontes sobre a precedência e a associatividade de Os operadores. A precedência controla quais operadores são executados primeiro em uma expressão. Tradição matemática e de programação (que remonta a antes do primeiro compilador Fortran) Em 1956, afirma-se que a multiplicação e a divisão têm precedência sobre a adição e a subtração, portanto $a+b*c$ significa $a+(b*c)$ e d/e significa $(d/e)-f$. Em qualquer gramática de expressões, Os operadores são agrupados em níveis de precedência, do mais baixo ao mais alto. O número total de níveis depende da linguagem. A linguagem C é notória por ter muitos níveis. níveis de precedência, num total de 15 níveis.

A associatividade controla o agrupamento de operadores no mesmo nível de precedência. Operadores Os operadores podem se agrupar à esquerda, por exemplo, abc em C significa $(ab)-c$, ou à direita, por exemplo, $a=b=c$ em C significa $a=(b=c)$. Em alguns casos, os operadores não se agrupam; por exemplo, Por exemplo, em Fortran , A.LE.B.LE.C é inválido.

Existem duas maneiras de especificar precedência e associatividade em uma gramática, implicitamente. e explicitamente. Até agora, especificamos isso implicitamente, usando não-terminais separados. símbolos para cada nível de precedência. Esta é uma maneira perfeitamente razoável de escrever uma gramática, e se o Bison não tivesse regras de precedência explícitas, seria a única maneira.

Mas o Bison também permite especificar a precedência explicitamente. Podemos adicionar essas linhas ao Seção de definição para indicar como resolver os conflitos:

```

%left '+' '-'
%esquerda '*' '/'
%nonassoc '|' UMINUS

%type <a> exp

%%
...
exp: exp '+' exp { $$ = newast('+', $1,$3); }
    | exp      exp { $$ = newast('-', $1,$3); }

```

```

| exp exp { $$ = newast('*', $1,$3); }
| exp '/' exp { $$ = newast('/', $1,$3); }
| '|' exp { $$ = newast('|', $2, NULL); }
| '(' exp ')' { $$ = $2; }
| '-' exp %prec UMINUS { $$ = newast('M', NULL, $2); }
| NÚMERO { $$ = novonum($1); }
;

```

Cada uma dessas declarações define um nível de precedência, com a ordem do %left,

As declarações %right e %nonassoc definem a ordem de precedência, da mais baixa para a mais alta.

mais alta. Eles dizem aos bisontes que + e - são associativos à esquerda e têm a menor precedência.

nível; * e / são associativos à esquerda e têm um nível de precedência mais alto; e | e UMINUS, um

pseudotokens que representam o sinal menos unário, não possuem associatividade e estão no nível mais alto.

precedência. (Não temos nenhum operador associativo à direita aqui, mas se tivéssemos, eles seriam

(use %right.) O Bison atribui a cada regra a precedência do token mais à direita no lado direito; se esse token não tiver

precedência atribuída, a regra não terá precedência sobre ela.

Quando o bisão se depara com um conflito de deslocamento/redução, ele consulta a tabela de precedência.

E se todas as regras envolvidas no conflito tiverem uma precedência atribuída, ela será usada.

para resolver o conflito.

Em nossa gramática, todos os conflitos ocorrem nas regras da forma exp OP exp, então configurando

A precedência dos quatro operadores permite que todos os conflitos sejam resolvidos. Este analisador sintático

Usar precedência resulta em um sistema ligeiramente menor e mais rápido do que aquele com regras adicionais para

precedência implícita, já que possui menos regras para reduzir.

A regra para negação inclui %prec UMINUS. O único operador nessa regra tem baixa precedência, mas -, qual

queremos que a negação unária tenha precedência maior que a multiplicação, e não menor. O %prec instrui o Bison a

usar a precedência de UMINUS para a negação.

esta regra.

Quando não usar regras de precedência

Você pode usar regras de precedência para corrigir qualquer conflito de deslocamento/redução que ocorra na gramática.

Essa geralmente é uma péssima ideia. Em gramáticas de expressões, a causa dos conflitos é fácil.

Para entender, o efeito das regras de precedência é claro. Em outras situações, as regras de precedência resolvem

problemas de deslocamento/redução, mas podem ser extremamente difíceis de compreender.

exatamente qual o efeito que elas têm na gramática.

Use precedência apenas em duas situações: em gramáticas de expressões e para resolver o

O conflito do "else pendente" nas gramáticas para construções linguísticas if/then/else. (Veja a seção ["IF/THEN/ELSE"](#)

[na página 185](#) para exemplos deste último.) Caso contrário, se possível,

Você deve corrigir a gramática para eliminar o conflito. Lembre-se de que conflitos significam que

O Bison não consegue criar um analisador sintático para uma gramática, provavelmente porque ela é ambígua. Isso significa

Existem várias interpretações possíveis para a mesma entrada, e o analisador sintático criado por Bison é um exemplo disso.

Escolha uma delas. Exceto nos dois casos anteriores, isso geralmente indica um problema.

na sua definição de idioma. Em alguns casos, se uma gramática for ambígua para o bisão, é

quase certamente ambíguo também para os humanos. Consulte [o Capítulo 8](#) para obter mais informações sobre

Encontrar e corrigir conflitos, além dos recursos avançados do Bison que permitem usar gramáticas ambíguas, se você realmente quiser.

Uma calculadora avançada

O exemplo final deste capítulo expande a calculadora para torná-la um compilador pequeno, porém relativamente realista. Adicionaremos variáveis nomeadas e atribuições; expressões de comparação (maior, menor, igual, etc.); controle de fluxo com if/then/else e while/do; funções internas e definidas pelo usuário; e um pouco de recuperação de erros. A versão anterior da calculadora não aproveitava muito a representação de expressões na AST (Árvore Sintática Abstrata), mas nesta, a AST é a chave para a implementação do controle de fluxo e das funções definidas pelo usuário.

Aqui está um exemplo de como definir uma função de usuário e, em seguida, chamá-la, usando uma função integrada como um dos argumentos:

```
> seja avg(a,b) = (a+b)/2;
Média definida >
média(3, sqrt(25)) 4
=
```

Como antes, começamos com as declarações, mostradas no [Exemplo 3-5](#).

Exemplo 3-5. Cabeçalho da calculadora avançada fb3-2.h

```
/*
 * Declarações para uma calculadora fb3-1 */

/* interface para o analisador léxico
 */ extern int yylineno; /* do analisador léxico */
void yyerror(char *s, ...);

/* tabela de símbolos */
struct symbol { char          /* um nome de variável */
               *name; double
               value; /*
               symlist *symlist para a função */ struct ast *func; struct
               de argumentos fictícios */ };

/* tabela simbólica simples de tamanho fixo
 */ #define NHASH 9997
struct symbol symtab[NHASH];

struct symbol *lookup(char*);

/* lista de símbolos, para uma lista de argumentos */
struct symlist { struct
               symbol *sym; struct
               symlist *next; };
```

```
struct symlist *newsymlist(struct symbol *sym, struct symlist *next); void
symlistfree(struct symlist *sl);
```

A tabela de símbolos foi adaptada daquela usada no capítulo anterior. Na calculadora, cada símbolo pode ser tanto uma variável quanto uma função definida pelo usuário. O campo `value` armazena o valor do símbolo como uma variável, o campo `func` aponta para a AST (Árvore Sintática Abstrata) do código da função definido pelo usuário, e `syms` aponta para uma lista encadeada de argumentos fictícios, que também são símbolos. (No exemplo anterior, `avg` é a função, e `a` e `b` são os argumentos fictícios.) As funções em C `newsymlist` e `symlistfree` criam e liberam esses símbolos.

```
/* tipos de nós * + -
 * / 0-7 operações
 *
 * de comparação, codificadas em bits: 04 igual, 02 menor, 01 maior
 * Menos lunar
 * Lista de expressões ou declarações em L
 * Declaração SE
 * Declaração W WHILE
 * Referência do
 * símbolo N = atribuição
 * Lista de símbolos
 * Chamada de função integrada F
 * Chamada de função de usuário em
C */

enum bifs {                                /* funções internas */
    B_sqrt = 1,
    B_exp,
    B_log,
    B_print };

/* nós na árvore de sintaxe abstrata */ /* todos
têm tipo de nó inicial comum */

struct ast { int
    nodetype;
    struct ast *l;
    struct ast *r; };

struct fncall { int                        /* função integrada */ /* tipo
    nodetype;                             F */
    struct ast *l;
    enum bifs functype; };

struct ufncall { int                     /* função do usuário
    nodetype;                             */ /* tipo C */ /
    struct ast *l;                       * lista de argumentos */
    struct symbol *s; };

struct fluxo {
```



```

int nodetype;           /* tipo I ou W */ /*
struct ast *cond;       condição */ /*
struct ast *tl;         ramificação then ou lista do */ /*
struct ast *el; };      ramificação else opcional */

struct numval
{ int nodetype;         /* tipo K */
  double number; };

struct symref { int
  nodetype;             /* tipo N */
  struct symbol *s; };

struct symasgn
{ int nodetype;         /* tipo = */
  struct symbol *s;
  struct ast *v; };      /* valor */

/* construir uma AST
*/ struct ast *newast(int nodetype, struct ast *l, struct ast *r); struct ast
*newcmp(int cmptype, struct ast *l, struct ast *r); struct ast *newfunc(int
functype, struct ast *l); struct ast *newcall(struct symbol
*s, struct ast *l); struct ast *newref(struct symbol *s); struct ast
*newasgn(struct symbol *s, struct ast *v);
struct ast *newnum(double d); struct ast *newflow(int nodetype,
struct ast *cond, struct ast *tl,
struct ast *tr);

/* define uma função */
void dodef(struct symbol *name, struct symlist *syms, struct ast *stmts);

/* avaliar uma AST */
double eval(struct ast *);

/* exclui e libera uma AST */ void
treefree(struct ast *);

/* interface para o analisador
léxico */ extern int yylineno; /* do analisador
léxico */ void yyerror(char *s, ...);

```

Esta versão possui consideravelmente mais tipos de nós na AST. Como antes, cada tipo de nó começa com um tipo de nó que o código de percurso na árvore pode usar para determinar o tipo de nó. O nó básico da AST também é usado para comparações, sendo que cada tipo de comparação (menor, menor ou igual a, igual a, etc.) é um tipo diferente, e para listas de expressões.

§ Este é um truque clássico de C que não funciona em C++. Se você escrever seu analisador sintático em C++, como descrevemos no [Capítulo 9](#), você terá que usar uma união explícita de estruturas dentro da estrutura AST para obter um resultado semelhante.

As funções internas possuem um nó `fncall` com a AST (Árvore Sintática Abstrata) do argumento (cada função interna recebe um argumento) e um enum que indica qual função interna é. Existem três funções padrão: `sqrt`, `exp` e `log`, além de `print`, uma função que imprime seu argumento e o retorna como valor. Chamadas para funções definidas pelo usuário possuem um nó `ufncall` com um ponteiro para a função, que é uma entrada na tabela de símbolos, e uma AST, que é uma lista dos argumentos.

As expressões de controle de fluxo `if/then/else` e `while/do` utilizam um nó de fluxo com a expressão de controle, o ramo `then` ou a lista `do`, e o ramo `else` opcional.

As constantes são do tipo `numval`, como antes; as referências a símbolos são do tipo `symref`, com um ponteiro para o símbolo na tabela de símbolos; e as atribuições são do tipo `symasn`, com um ponteiro para o símbolo a ser atribuído e a AST (Árvore Sintática Abstrata) do valor a ser atribuído a ele.

Cada AST possui um valor. O valor de um `if/then/else` é o valor do ramo escolhido; o valor de um `while/do` é o último valor da lista do tipo `do`; e o valor de uma lista de expressões é a última expressão. Finalmente, temos procedimentos em C para criar cada tipo de nó da AST e um procedimento para criar uma função definida pelo usuário.

Analizador de calculadora avançada

[O exemplo 3-6](#) mostra o analisador sintático para a calculadora AST avançada.

Exemplo 3-6. Analisador de calculadora avançada `fb3-2.y`

```
/* calculadora com AST */

%
{ #include <stdio.h> #include
<stdlib.h> #include "fb3-2.h"
%}

%union
{ struct ast *a;
  double d;
  struct symbol *s;          /* qual símbolo */
  struct symlist *sl; int    /* qual função */
  fn; }

/* declarar tokens */
%token <d> NUMBER
%token <s> NAME
%token <fn> FUNC
%token EOL
```

Este projeto é vagamente baseado na antiga linguagem de programação de sistemas BLISS. Seria perfeitamente possível projetar uma AST que tratasse expressões e instruções separadamente, mas a implementação é um pouco mais simples desta forma.

```
%token SE ENTÃO SENÃO ENQUANTO FAÇA DEIXE
```

```
%nonassoc <fn> CMP
```

```
%right '='
```

```
%left '+' '-' %left
```

```
'*' '/' %nonassoc
```

```
'|' UMINUS
```

```
%type <a> lista de instruções exp lista exp
```

```
%type <sl> lista simbólica
```

```
%início da lista de cálculos
```

```
%%
```

A %union aqui define vários tipos de valores de símbolos, o que é típico em analisadores sintáticos Bison realistas. Além de um ponteiro para uma AST e um valor numérico, um valor pode ser um ponteiro para a tabela de símbolos de um símbolo do usuário, uma lista de símbolos ou um subtipo de um token de comparação ou função. (Usamos a palavra "símbolo" de forma um tanto confusa aqui, tanto para nomes usados na gramática Bison quanto para nomes que o usuário digita no programa compilado. Usaremos "símbolo do usuário" para este último caso quando o contexto não for claro.)

Há um novo token FUNC para as funções internas, cujo valor indica qual função, e seis palavras reservadas, de IF a LET. O token CMP representa qualquer um dos seis operadores de comparação, cujo valor indica qual operador. (Esse truque de usar um token para vários operadores sintaticamente semelhantes ajuda a reduzir o tamanho da gramática.)

A lista de declarações de precedência começa com os novos operadores CMP e = .

Uma declaração %start identifica a regra de nível superior, portanto não precisamos colocá-la no início do analisador sintático.

Sintaxe da instrução da calculadora

```
declaração: SE exp ENTÃO lista { $$ = newflow('I', $2, $4, NULL); }
           | SE exp ENTÃO lista SENÃO lista { $$ = newflow('I', $2, $4, $6); }
           | ENQUANTO exp FAÇA lista { $$ = novofluxe('W', $2, $4, NULL); } | exp
;

lista: /* nada */ { $$ = NULL; }
     | stmt ';' lista { se ($3 == NULL) $$ = $1; senão $
                        $ =
                        newast('L', $1, $3);
                       }
;

```

Nossa gramática distingue entre proposições (stmt) e expressões (exp). Uma proposição é um fluxo de controle (se/então/senão ou enquanto/faça) ou uma expressão. As proposições se e enquanto recebem listas de proposições, com cada proposição na lista sendo seguida por uma condição.

por um ponto e vírgula. Cada regra que corresponde a uma declaração chama uma rotina para construir um nó AST apropriado.

O design da sintaxe aqui é em grande parte arbitrário, e essa é uma das vantagens de usar...

A grande vantagem do Bison em criar um analisador sintático é a facilidade com que se pode experimentar variações. A interação

Entre os detalhes da sintaxe, as nuances podem ser bastante sutis; se, por exemplo, a definição de lista tivesse

Se os pontos e vírgulas fossem colocados entre as declarações em vez de depois delas, a gramática ficaria ambígua, a menos que também fossem adicionados os marcadores de fechamento FI e ENDDO para indicar o fim de instruções if/then e while/do.

A definição de lista é recursiva à direita, isto é, stmt ; lista em vez de lista stmt ;

Não faz diferença alguma em relação ao idioma reconhecido, mas facilita a construção.

a lista de declarações encadeadas da cabeça para a cauda, em vez da cauda para a cabeça. Cada vez a regra de lista stmt; é reduzida, ela cria um link que adiciona a declaração ao cabeçalho de a lista até agora. Se a regra fosse list stmt ;, a declaração precisaria ir no final da lista, o que exigiria uma lista circularmente encadeada mais complexa ou então invertendo a lista no final (como fizemos com a lista de referências no [Capítulo 1](#)).

Uma desvantagem da recursão à direita em relação à recursão à esquerda é que a recursão à direita coloca tudo em prática. das instruções ainda não reduzidas na pilha do analisador sintático e, em seguida, reduz todas elas em o final da lista, enquanto a recursão à esquerda constrói a lista uma instrução por vez como entrada. é analisado. Em uma situação como essa, onde é improvável que a lista tenha mais do que alguns itens.

Não importa o tamanho, mas em um idioma onde a lista pode ter milhares de itens...

itens, vale a pena criar a lista com uma regra recursiva à esquerda e depois invertê-la para evitar

estouro de pilha do analisador sintático. Alguns programadores também acham a recursão à esquerda mais fácil de depurar.

pois tende a gerar resultados após cada instrução, em vez de todos de uma vez no final.

Sintaxe de expressão da calculadora

```
exp: exp CMP exp          { $$ = newcmp($2, $1, $3); }
    | exp '+' exp { $$ = newast('+', $1,$3); }
    | exp { '$$ = newast('-', $1,$3); }
    | exp { '$$ = newast('*', $1,$3); }
    | exp '/' exp { '$$ = newast('/', $1,$3); }
    | '(' exp { '$$ = newast('(', $2, NULL); }
    | '(' exp ')' { '$$ = $2; }
    | '-' exp %prec UMINUS { '$$ = newast('M', $2, NULL); }
    | NÚMERO { '$$ = novonum($1); }
    | NOME { '$$ = newref($1); }
    | NOME '=' { '$$ = newasgn($1, $3); }
    | FUNC '(' explista ')' { '$$ = novafunc($1, $3); }
    | NOME '(' explist ')' { '$$ = newcall($1, $3); }
    ;
```

lista de explicações: exp

```
| exp ',' explist { '$$ = newast('L', $1, $3); }
;
lista_simbólica: NOME { '$$ = nova_lista_simbólica($1, NULL); }
| NOME ',' lista_simbólica { '$$ = nova_lista_simbólica($1, $3); }
;
```

A sintaxe de expressão é uma versão ligeiramente expandida da sintaxe de expressão do exemplo anterior. Uma nova regra para CMP lida com os seis operadores de comparação, usando o valor do CMP para determinar qual operador foi utilizado, e uma regra para atribuições cria um nó de atribuição.

Existem regras separadas para funções internas identificadas por um nome reservado (FUNC) e funções de usuário identificadas por um símbolo de usuário (NAME).

Uma regra para `explist`, uma lista de expressões, constrói uma AST (Árvore Sintática Abstrata) das expressões usadas como argumentos reais em uma chamada de função. Uma regra separada para `symlist`, uma lista de símbolos, constrói uma lista encadeada de símbolos para os argumentos fictícios em uma definição de função. Ambas são recursivas à direita para facilitar a construção da lista na ordem desejada.

Gramática da calculadora de alto nível

```
calclist: /* nada */ | calclist stmt
        EOL { printf("= %4.4g\n> ",
            eval($2)); treefree($2); }

| calclist LET NAME '(' symlist ')' '=' list EOL { dodef($3, $5, $8);
        printf("Definido %s\n> ", $3-
            >name); }

| erro de lista de cálculos EOL { yyerrok; printf("> "); } ;
```

A última parte da gramática é o nível superior, que reconhece uma lista de instruções e declarações de funções. Como antes, o nível superior avalia a AST em busca de uma instrução, imprime o resultado e, em seguida, libera a AST. Uma definição de função é simplesmente salva para uso futuro.

Recuperação Básica de Erros do

Analizador Sintático A última regra do analisador sintático oferece uma pequena possibilidade de recuperação de erros. Devido ao funcionamento dos analisadores sintáticos do Bison, raramente vale a pena o esforço de tentar corrigir erros, mas é possível, pelo menos, recuperar o analisador para um estado em que ele possa continuar. O pseudo-token de erro especial indica um ponto de recuperação de erro. Quando um analisador sintático do Bison encontra um erro, ele começa a descartar símbolos da pilha de análise até atingir um ponto em que um token de erro seria válido; em seguida, ele descarta tokens de entrada até encontrar um que possa ser deslocado em seu estado atual e, então, continua a análise a partir daí. Se a análise falhar novamente, ele descarta mais símbolos da pilha e tokens de entrada até que possa retomar a análise ou a pilha esteja vazia e a análise falhe. Para evitar uma cascata de mensagens de erro enganosas, o analisador normalmente suprime quaisquer mensagens de erro de análise após a primeira até que tenha deslocado com sucesso três tokens em sequência. A macro `yyerrok` em uma ação informa ao analisador sintático que a recuperação foi concluída, portanto, as mensagens de erro subsequentes serão produzidas.

Embora seja possível, em princípio, adicionar muitas regras de erro para tentar realizar uma ampla recuperação de erros, na prática é raro ter mais de uma ou duas regras de erro. O token de erro

é quase sempre usado para ressincronizar em um caractere de pontuação em uma regra recursiva de nível superior, como fazemos aqui.

Se os símbolos descartados da pilha durante a recuperação de erros tiverem valores que apontam para armazenamento alocado, o processo de recuperação de erros causará vazamento de memória, já que os valores descartados nunca são liberados. Neste exemplo, não nos preocupamos com isso, mas o Bison oferece um recurso para instruir o analisador sintático a chamar seu código para liberar os valores descartados, conforme descrito no [Capítulo 6](#).

O Analisador Lexar Avançado de Calculadoras

Exemplo 3-7. Calculadora avançada léxica fb3-2.l

```
/* reconhecer tokens para a calculadora */ %option noyywrap
nndefault yylineno %{ #include "fb3-2.h" #include

"fb3-2.tab.h" %}

/* expoente de ponto flutuante
EXP      */ ([Ee][+-]?[0-9]+)

%%

/* operações de caractere único */ "+" |
"_" | "++"
| "/" |
"=" |
"|" | " "
| ":" |
"(" | ")"

{ return yytext[0]; }

/* operações de comparação, todas são um token CMP */
">" { yylval.fn = 1; return CMP; } { yylval.fn = 2;
"<" { return CMP; } { yylval.fn = 3; return CMP; }
"<=" { yylval.fn = 4; return CMP; } { yylval.fn = 5;
"==" { return CMP; } { yylval.fn = 6; return CMP; }
">="
"<="

/* palavras-chave */

"se" { retorne SE; } "então" { retorne
ENTÃO; } "senão" { retorne SENÃO; }
"enquanto" { retorne ENQUANTO; } "faça"
{ retorne FAÇA; } "deixe" { retorne DEIXE; }
```

```

/* funções internas */ "sqrt"
{ yyval.fn = B_sqrt; return FUNC; } "exp" { yyval.fn
= B_exp; return FUNC; } "log" { yyval.fn = B_log;
return FUNC; } "print" { yyval.fn = B_print; return
FUNC; }

/* nomes */
[a-zA-Z][a-zA-Z0-9]* { yyval.s = lookup(yytext); return NAME; }

[0-9]+ "." [0-9]* {EXP}? | "."?
[0-9]+ {EXP}? { yyval.d = atof(yytext); return NUMBER; }

"//".*
[ \t] /* ignorar espaços em branco */

\\n { printf("c> "); } /* ignorar continuação de linha */

\\n { return EOL; }

.
{ yyerror("Caractere misterioso %c\\n", *yytext); }
%%

```

O analisador léxico, mostrado no [Exemplo 3-7](#), adiciona algumas novas regras ao exemplo anterior. Há alguns novos operadores de caractere único. Os seis operadores de comparação retornam um token CMP com um valor léxico para distingui-los.

As seis palavras-chave e as quatro funções internas são reconhecidas por padrões literais. Observe que elas precisam preceder o padrão geral para que a correspondência com um nome seja priorizada em relação ao padrão geral. O padrão de nome busca o nome na tabela de símbolos e retorna um ponteiro para o símbolo.

Como antes, uma nova linha (EOL) marca o fim de uma string de entrada. Como uma função ou expressão pode ser muito longa para ser digitada em uma única linha, permitimos linhas de continuação. Uma nova regra do analisador léxico reconhece uma barra invertida e uma nova linha e não retorna nada ao analisador sintático, tornando a continuação invisível para ele. Mas exibe um aviso para o usuário.

Palavras Reservadas

Nessa gramática, as palavras `if`, `then`, `else`, `while`, `do`, `let`, `sqrt`, `exp`, `log` e `print` são reservadas e não podem ser usadas como símbolos definidos pelo usuário. A decisão de permitir que os usuários usem o mesmo nome para duas coisas no mesmo programa é discutível. Por um lado, isso pode dificultar a compreensão dos programas, mas, por outro lado, os usuários são forçados a inventar nomes que não entrem em conflito com os nomes reservados.

Ambos os casos podem ser levados ao extremo. O COBOL possui mais de 300 palavras reservadas, então ninguém consegue memorizá-las todas, e os programadores recorrem a convenções estranhas, como começar o nome de cada variável com um dígito para garantir que não haja conflito com uma palavra reservada. Por outro lado, o PL/I não possui palavras reservadas, então você pode escrever o seguinte:

```
SE SE = ENTÃO ENTÃO SENÃO = ENTÃO; SENÃO SENÃO = SE;
```

Os analisadores sintáticos Bison são muito mais fáceis de escrever se você reservar as palavras-chave; caso contrário, você precisa projetar cuidadosamente sua linguagem para que, em cada ponto em que o analisador léxico estiver lendo um token, um nome ou uma palavra-chave seja válido, mas não ambos, e você precisa fornecer um feedback extenso ao analisador léxico para que ele saiba qual usar. Tendo escrito analisadores léxicos para Fortran, que não possui palavras reservadas e (em suas versões clássicas) ignora todos os espaços em branco, recomendo fortemente que você reserve suas palavras-chave.

Construindo e Interpretando ASTs Finalmente,

temos o arquivo de código auxiliar, mostrado no [Exemplo 3-8](#). Parte deste arquivo é igual ao exemplo anterior; as funções main e yyerror permanecem inalteradas e não são repetidas aqui.

O código principal constrói e avalia as ASTs (Árvores de Sintaxe Abstrata). Primeiro, temos o gerenciamento da tabela de símbolos, que deve ser familiar a partir dos exemplos do [Capítulo 2](#).

Exemplo 3-8. Funções auxiliares avançadas da calculadora fb3-2func.c

```
/*
 * funções auxiliares para fb3-2 */

#include <stdio.h> #include
<stdlib.h> #include <stdarg.h>
#include <string.h>

#include <math.h>
#include "fb3-2.h"

/* tabela de símbolos
 */ /* hash de um
 símbolo */ static
unsigned symhash(char

*sym) { unsigned int hash =
    0; unsigned c;

    enquanto(c = *sym++) hash = hash*9 ^ c;

    retornar hash; }

struct symbol *
lookup(char* sym)

{ struct symbol *sp = &syntab[symhash(sym)%NHASH]; int
    scount = NHASH; /* quantos analisamos */

    while(--scount >= 0) { if(sp-
        >name && strcmp(sp->name, sym)) { return sp; }

        if(!sp->name) { sp- /* nova entrada */
            >name = strdup(sym);
```



```

sp->value = 0; sp-
>func = NULL; sp-
>syms = NULL; return
sp; }

```

```

if(++sp >= symtab+NHASH) sp = symtab; /* tente a próxima entrada */ } yyerror("estouro

```

```

da tabela de símbolos\n"); abort(); /* tentei todas,
a tabela está cheia */

```

```

}

```

Em seguida, vêm os procedimentos para construir os nós da AST e as listas simbólicas. Todos eles alocam um nó e preenchem os campos adequadamente para o tipo de nó. Uma versão estendida do `treefree` percorre recursivamente uma AST e libera todos os nós da árvore.

```

struct ast *
newast(int nodetype, struct ast *l, struct ast *r) { struct ast *a =

malloc(sizeof(struct ast));

if(!a)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= nodetype; a->l = l; a->r = r;
return a; }

```

```

struct ast *
newnum(double d)

{ struct numval *a = malloc(sizeof(struct numval));

if(!a)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= 'K'; a->number = d;
return (struct ast
*)a; }

```

```

struct ast *
novocmp(int cmptype, struct ast *l, struct ast *r) { struct ast *a =

malloc(sizeof(struct ast));

if(!a)
{ yyerror("sem espaço"); exit(0); }

```

```

a->nodetype = '0' + cmptype; a->l = l; a-
>r = r; return
a; }

```

```

struct ast *
newfunc(int functype, struct ast *l) { struct fncall *a

= malloc(sizeof(struct fncall));

if(!a)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= 'F'; a->l = l; a->functype
= functype;
return (struct ast *)a; }

```

```

struct ast *
newcall(struct symbol *s, struct ast *l) { struct ufncall *a =

malloc(sizeof(struct ufncall));

if(!a)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= 'C'; a->l = l; a->s = s;
return (struct
ast *)a; }

```

```

struct ast *
newref(struct symbol *s) { struct

symref *a = malloc(sizeof(struct symref));

if(!a)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= 'N'; a->s = s; return
(struct ast
*)a; }

```

```

struct ast *
newasgn(struct symbol *s, struct ast *v) { struct symasgn

*a = malloc(sizeof(struct symasgn));

```

```

if(la)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= '='; a->s = s; a->v = v;
return (struct
ast *)a; }

struct ast *
newflow(int nodetype, struct ast *cond, struct ast *tl, struct ast *el) { struct flow *a = malloc(sizeof(struct
flow));

if(la)
{ yyerror("sem espaço"); exit(0); }
a->nodetype

= nodetype; a->cond = cond; a-
>tl = tl; a->el = el;
return (struct
ast *)a; }

/* libera uma árvore de ASTs */ void

treefree(struct ast *a) { switch(a-
>nodetype) {

/* duas subárvores */
caso '+':
caso '-':
caso '*':
caso '/': caso
'1': caso '2': caso '3': caso '4': caso '5': caso '6': caso 'L': treefree(a->r);

/* uma subárvore */ caso
'l': caso 'M':
caso 'C': caso 'F':
treefree(a->l);

/* nenhuma subárvore
*/ caso 'K': caso 'N':
interromper;

caso '=':
free( ((struct symasgn *)a)->v); break;

/* até três subárvores */

```

```

    caso 'I': caso 'W':
        free( ((struct flow *)a)->cond); se( ((struct flow
        *)a)->t) treefree( ((struct flow *)a)->t); se( ((struct flow *)a)->el) treefree( ((struct flow *)a)-
        >el); break;

    padrão: printf("erro interno: liberar nó inválido %c\n", a->nodetype); }

    free(a); /* sempre libera o próprio nó */ }

struct symlist *
newsymlist(struct symbol *sym, struct symlist *next) { struct symlist *sl =

    malloc(sizeof(struct symlist));

    if(!sl)
        { yyerror("sem espaço"); exit(0); }
    sl->sym =

    sym; sl->next =
    next; return sl; }

/* libera uma lista de símbolos */ void

symlistfree(struct symlist *sl) { struct symlist

    *nsl;

    while(sl) { nsl =
        sl->next; free(sl); sl =
        nsl; } }

```

O núcleo da calculadora é o `eval`, que avalia uma AST construída no analisador sintático. Seguindo a prática em C, as comparações retornam 1 ou 0 dependendo se a comparação for bem-sucedida, e os testes em if/then/else e while/do tratam qualquer valor diferente de zero como verdadeiro.

Para expressões, usamos a conhecida busca em profundidade na árvore para calcular o valor. Uma AST facilita a implementação de estruturas if/then/else: avaliamos a AST da condição para decidir qual caminho seguir e, em seguida, avaliamos a AST para determinar o caminho a ser percorrido. Para avaliar laços while/do, um laço em eval avalia a AST da condição e, em seguida, a AST do corpo, repetindo o processo enquanto a AST da condição permanecer verdadeira. Qualquer AST que faça referência a variáveis que são alteradas por uma atribuição terá um novo valor a cada avaliação.

```

static double callbuiltin(struct fncall *); static double
calluser(struct ufncall *);

dobro

```

```

eval(struct ast *a)

{ double v;

  if(!a)
    { yyerror("erro interno, avaliação nula"); return
      0.0; }

  switch(a->nodetype) { /*
    constante */ case
    'K': v = ((struct numval *)a)->number; break;

    /* referência de nome */
    caso 'N': v = ((struct symref *)a)->s->value; break;

    /* atribuição */
    caso '=': v = ((struct symasn *)a)->s->value = eval(((struct
      symasn *)a)->v); break;

    /* expressões */ case
    '+': v = eval(a->l) + eval(a->r); break; case '-': v = eval(a->l)
    - eval(a->r); break; case '*': v = eval(a->l) * eval(a->r);
    break; case '/': v = eval(a->l) / eval(a->r); break; case '!': v
    = fabs(eval(a->l)); break; case 'M': v = -eval(a->l); break;

    /* comparações */
    case '1': v = (eval(a->l) > eval(a->r)) ? 1 : 0; break; case '2': v = (eval(a-
    >l) < eval(a->r)) ? 1 : 0; break; case '3': v = (eval(a->l) != eval(a->r)) ? 1 :
    0; break; case '4': v = (eval(a->l) == eval(a->r)) ? 1 : 0; break; case '5': v
    = (eval(a->l) >= eval(a->r)) ? 1 : 0; break; case '6': v = (eval(a->l) <=
    eval(a->r)) ? 1 : 0; break;

    /* fluxo de controle */ /*
    expressões nulas são permitidas na gramática, portanto, verifique-as */

    /* if/then/else */ case 'I':
    if( eval( ((struct
      flow *)a)->cond) != 0 ) { verificar a condição if( ((struct flow *)a)->tl) { o ramo verdadeiro

      v = eval( ((struct flow *)a)->tl); } else v =
      0.0; }
      else
      /* um valor padrão */
      { if( ((struct
        flow *)a)->el) {
          v = eval(((struct flow *)a)->el); } else v = 0.0;
          o ramo falso

          /* um valor padrão */

        } quebrar;

      /* enquanto/faça */

```

```

    caso 'W': v
        = 0.0; /* um valor padrão */

    se( ((struct flow *)a)->tl) {
        while( eval(((struct flow *)a)->cond) != 0) avalie a condição avalie as instruções de destino
            v = eval(((struct flow *)a)->tl);

    } quebrar; /* o valor da última instrução é o valor do while/do */

/* lista de instruções */ case 'L':
    eval(a->l); v = eval(a->r); break;

    caso 'F': v = callbuiltin((struct fncall *)a); break;

    caso 'C': v = calluser((struct ufncall *)a); break;

    default: printf("erro interno: nó inválido %c\n", a->nodetype); } return v; }

```

Avaliando funções na calculadora

As partes mais complexas do código no avaliador lidam com funções. As funções internas são relativamente simples: elas determinam qual função é e chamam o código específico para executá-la.

```

estático duplo
callbuiltin(struct fncall *f) { enum bifs

    functype = f->functype; double v = eval(f->l);

    switch(functype) { case
        B_sqrt: return
            sqrt(v); case B_exp:
            return exp(v);
        case B_log: return
            log(v); case
            B_print: printf("=
                %4.4g\n", v);
            return v; default:

                yyerror("Função interna desconhecida %d", functype); return 0.0; } }

```

Funções definidas pelo usuário

A definição de uma função consiste no nome da função, uma lista de argumentos fictícios e uma AST (Árvore Sintática Abstrata) que representa o corpo da função. Definir a função simplesmente salva

A lista de argumentos e a AST (Árvore Sintática Abstrata) na entrada da tabela de símbolos da função, substituindo qualquer versão anterior.

```
/* define uma função */ void

typedef(struct symbol *name, struct symlist *syms, struct ast *func) { if(name->syms)

symlistfree(name->syms); if(name->func) treefree(name->func); name->syms = syms; name->func = func; }
```

Suponha que você defina uma função para calcular o máximo de seus dois argumentos:

> seja $\max(x,y)$ = se $x \geq y$ então x ; senão y ; > $\max(4+5,6+7)$

A função possui dois argumentos fictícios, x e y . Quando a função é chamada, o avaliador faz o seguinte:

1. Avalie os argumentos reais, $4+5$ e $6+7$ neste caso.
2. Salve os valores atuais dos argumentos fictícios e atribua a eles os valores dos argumentos reais.
3. Avalie o corpo da função, que agora usará os valores reais dos argumentos quando se referir aos argumentos fictícios.
4. Restaure os valores antigos das variáveis fictícias.
5. Retorne o valor da expressão do corpo.

O código para fazer isso conta os argumentos, aloca dois arrays temporários para os valores antigos e novos dos argumentos fictícios e, em seguida, executa as etapas descritas anteriormente.

```
static double
calluser(struct ufncll *f) {

    struct symbol *fn = f->s; struct          /* nome da função */ /*
    symlist *sl; struct ast *args            argumentos fictícios */ /*
    = f->l; double *oldval, *newval;          argumentos reais */ /* valores
    double v; int nargs; int i;              de argumentos salvos */

    if(!fn->func)
        { yyerror("chamada para função indefinida", fn->name); return 0; }

    /* contar os argumentos */ sl = fn->
    >syms; for(nargs =
    0; sl; sl = sl->next) nargs++;
```

```

/* prepare-se para salvá-los */
oldval = (double *)malloc(nargs * sizeof(double)); newval =
(double *)malloc(nargs * sizeof(double)); if(!oldval || !newval) {

    yyerror("Sem espaço em %s", fn->name); return 0.0; }

/* avaliar os argumentos */ for(i = 0;
i < nargs; i++) { if(!args)

    { yyerror("poucos argumentos na chamada para %s", fn->name);
      free(oldval); free(newval); return
        0.0; }

    if(args->nodetype == 'L') { /* se este for um nó de lista */ newval[i] =
      eval(args->l); args = args->r; }
      else { newval[i] =
        eval(args); /* se for o fim da lista */
          args = NULL; } }

/* salva os valores antigos das variáveis fictícias, atribui os
novos */ sl = fn-
>syms; for(i = 0; i < nargs; i++)
  { struct symbol *s = sl->sym;

    oldval[i] = s->value; s-
    >value = newval[i]; sl = sl-
    >next; }

livre(novovalor);

/* avaliar a função */ v = eval(fn-
>func);

/* retorna os valores reais das variáveis fictícias */ sl = fn-
>syms; for(i = 0;
i < nargs; i++) { struct symbol *s =
  sl->sym;

  s->value = oldval[i]; sl = sl-
  >next; }

liberar(valor
antigo);
retornar v; }

```


Usando a calculadora avançada

Esta calculadora é suficientemente flexível para realizar alguns cálculos úteis. [O exemplo 3-9](#) mostra a função `sq` para calcular raízes quadradas iterativamente usando o método de Newton, bem como uma função auxiliar `avg` para calcular a média de dois números.

Exemplo 3-9. Calculando raízes quadradas com a calculadora

```
> seja sq(n)=e=1; enquanto |(t=n/e)-e|>.001 faça e=avg(e,t);; Definido sq >
seja
avg(a,b)=(a+b)/2; Definido avg
> sq(10) =
3.162 >
sqrt(10)
= 3.162 >
sq(10)-
sqrt(10) = 0.000178
```

precisão melhor que o limite de 0,001

Exercícios

1. Experimente algumas variantes da sintaxe na calculadora aprimorada. No exemplo anterior, a função `sq` precisa terminar com dois pontos e vírgulas para fechar tanto o laço `while` quanto a instrução `let`, o que é bastante desajeitado. Você consegue alterar a sintaxe para torná-la mais intuitiva? Se você adicionar símbolos de fechamento às instruções condicionais `if/then/else/fi` e aos laços `while/do/done`, consegue tornar a sintaxe das listas de instruções mais flexível?
2. No último exemplo, as funções do usuário avaliam todos os argumentos reais, colocam-nos em um array temporário e, em seguida, os atribuem aos argumentos fictícios. Por que não fazer isso um de cada vez e definir os argumentos fictícios à medida que os argumentos reais são avaliados?